

```

QPushButton *button;
QVBoxLayout *vbox;

/* Create the vbox, QLineEdit and button objects */
vbox = new QVBoxLayout(&w);
line = new QLineEdit(&w);
button = new QPushButton(&w);
button->setText("Click Me");

/* Create a new size policy and apply to button widget */
QSizePolicy policy;
policy.setHorizontalPolicy(QSizePolicy::Fixed);
policy.setVerticalPolicy(QSizePolicy::Fixed);
button->setSizePolicy(policy);

/* Add widgets to VBox */
vbox->addWidget(line);
vbox->addWidget(button);

w.show();
return a.exec();
}

```

The final aspect is the stretching of the widgets placed in a `BoxLayout`.

As shown in Figure 3, the `lineEdit` and `pushbutton` widgets are packed in a `QVBoxLayout` and the default sizes of the `lineEdit` and `pushbutton` are different (basically, the width). But due to the `boxlayout` the `pushbutton` gets stretched to the width of the `lineEdit`. The look of the application becomes awkward. To avoid this, you can apply the size policy on the `pushbutton`. The `QSizePolicy` class will do it for you.

```

QSizePolicy policy;
policy.setHorizontalPolicy(QSizePolicy::Fixed);
policy.setVerticalPolicy(QSizePolicy::Fixed);
button->setSizePolicy(policy);

```

In the above code, a size policy has been created with a `Fixed` horizontal and vertical size. Then you have set this policy for the `pushbutton`. The output is shown in Figure 4.

Now even if you stretch the application window, the `pushbutton` will not be stretched.

Stretch factor

Sometimes, you may need to stretch the widgets in a `boxlayout`, but with a different ratio. For example, one widget should be stretched to double the other. In this case, you can specify the stretch factor while adding the widget to the `BoxLayout`:

```

hbox->addWidget(line1, 1);
hbox->addWidget(button, 2);
hbox->addWidget(line2, 3)

```

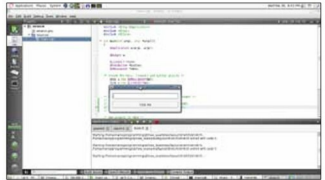


Figure 3: Stretched pushbutton



Figure 4: Normal pushbutton

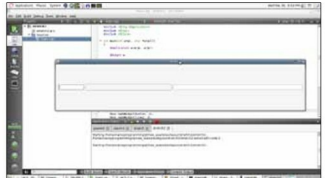


Figure 5: Stretch factor

The above code uses one more parameter in the `addWidget()` function—the stretch factor. These three widgets will now be laid out in the ratio of 1:2:3 of the total space provided by `BoxLayout`, as shown in Figure 5.

There are so many things that you can try, e.g., the grid layout in which the widgets are placed in a matrix form, `formlayout`, `splitters`, etc. Go through the Qt documentation for the details. **END** 

By: Manoj Kumar

The author is a freelance developer and trainer. He leads a team in Linux kernel programming, Linux administration, cluster computing, embedded systems and QT/GTK programming on Linux. You can reach him at manoj@nixitsolutions.com and read his blog, www.open4india.blogspot.com, to share your Linux knowledge.